
VIKRANT DASHBOARD

Master Engineering Reference

The complete technical authority document for ownership, troubleshooting, and advancement

PART 1 WSL & Infrastructure	PART 2 Django Internals	PART 3 Database Mastery	PART 4 Redis & Messaging
PART 5 Celery Architecture	PART 6 WebSockets & Channels	PART 7 Frontend Engineering	PART 8 Security & Hardening
PART 9 Performance & Scaling	PART 10 Troubleshooting Bible	PART 11 Upgrade Playbooks	PART 12 System Ownership

How to use this document

This is a reference document, not a tutorial. Every section answers the question a production engineer asks when something breaks, needs upgrading, or needs explaining to a new team member. Read it cover to cover once. After that, use it as a lookup. The Troubleshooting Bible (Part 10) is designed for crisis navigation — bookmark it.

YOUR GOAL

When anyone in your organisation has a question about this system — infrastructure, bugs, capacity, security, upgrades — you are the person they come to. This document is the foundation of that expertise. It covers not just what things are, but why they exist, what breaks them, and how to fix them.

PART
1

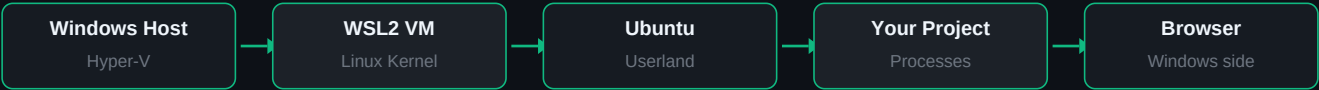
WSL & Infrastructure

The operating environment that runs everything

MASTER REFERENCE

1.1 WSL2 Architecture & How It Differs from a Real Linux Server

WSL2 runs a real Linux kernel inside a lightweight Hyper-V VM. Your Windows filesystem is mounted at `/mnt/c/`. Network traffic goes through a virtual network adapter. Understanding this architecture explains several WSL-specific behaviours.



WSL2 Property	What You Need to Know
WSL2 IP	Dynamic — changes on every reboot. Use 127.0.0.1 or localhost for local services
Filesystem speed	Linux paths (<code>/home/</code>) are fast. Windows paths (<code>/mnt/c/</code>) are 5-10x slower. Keep project on <code>/home/</code>
Service startup	<code>systemd</code> not enabled by default. Use: <code>sudo service <name> start</code>
Port forwarding	Services on WSL port 8000 are accessible on Windows at <code>localhost:8000</code> automatically
Memory limit	WSL2 uses up to 50% of RAM by default. Set in <code>~/.wslconfig</code> : <code>[wsl2] memory=4GB</code>
Process isolation	WSL2 processes are invisible to Windows Task Manager. Use <code>htop</code> inside WSL
File permissions	Windows files in <code>/mnt/c/</code> have no real permissions — always work in <code>/home/</code>
Clock sync	WSL2 clock can drift after Windows sleep. Fix: <code>sudo hwclock -s</code>

Permanent WSL Configuration

```

bash

# ~/.wslconfig (Windows side: C:\Users\YourName\.wslconfig)
# Controls WSL2 VM resources
[wsl2]
memory=6GB          # cap RAM usage
processors=4         # cap CPU cores
swap=2GB
localhostForwarding=true

# Apply: restart WSL from PowerShell: wsl --shutdown

# ~/.bashrc additions (WSL Ubuntu side)
export PROJ="$HOME/vikrant/VikrantDashboard"
export PGPASSWORD="your-db-password"      # so psql doesn't prompt
export PYTHONDONTWRITEBYTECODE=1         # no __pycache__ clutter
export PYTHONUNBUFFERED=1                 # force stdout flush (critical for LogCapture)

# Service aliases
alias start-services="sudo service postgresql start && sudo service redis-server start"
alias stop-services="sudo service postgresql stop && sudo service redis-server stop"
alias proj="cd $PROJ && source venv/bin/activate"

# Project health at a glance
alias vk-status="pg_isready -h 127.0.0.1 && redis-cli ping && ss -tlnp | grep -E '8000|5555'"

```

1.2 Service Management — PostgreSQL & Redis in WSL

IMPORTANT

WSL2 does not auto-start services on boot. You must start PostgreSQL and Redis every time you open a new WSL session. Add start-services to your workflow or enable systemd (WSL 0.67.6+) for automatic startup.

[illegible]

1.3 Environment Isolation & The Virtual Environment Contract

The venv is not just a convenience. It is a **contract**: any developer who clones the repo and runs `pip install -r requirements.txt` gets the exact same dependency tree. Break this contract and you break reproducibility.

NEVER DO THIS

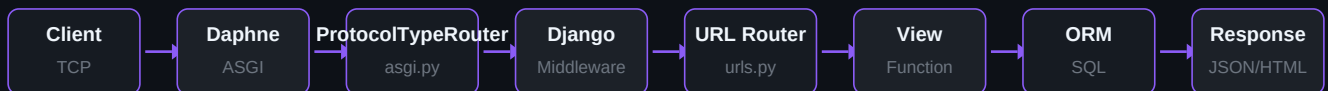
Vikrant Dashboard — Master Engineering Reference | WSL Edition

Django Internals

Request lifecycle, middleware, ORM, and ASGI deep dive

MASTER REFERENCE

2.1 The Complete Request Lifecycle



Step	What Happens
1. TCP connection	Client opens socket to port 8000. Daphne accepts it.
2. Protocol detect	Daphne reads HTTP/1.1 or WebSocket upgrade header.
3. ASGI dispatch	ProtocolTypeRouter in asgi.py routes http→Django, websocket→Channels.
4. Middleware chain	Each middleware wraps the request. Order in MIDDLEWARE list matters.
5. URL resolution	ROOT_URLCONF → urlpatterns list → first matching path()
6. View execution	Django calls the view function with the request object.
7. ORM queries	View reads/writes PostgreSQL via Django ORM (psycopg2 driver).
8. Response	View returns JsonResponse or render(). Middleware wraps on exit.

Middleware Execution Order — Critical to Understand

Middleware wraps like an onion. MIDDLEWARE list top→bottom on request, bottom→top on response. A slow middleware multiplies its cost across every request.

```

Python

# settings.py — MIDDLEWARE order matters
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",      # 1st in, last out
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware", # last in, 1st out
]

# Custom request timing middleware
import time, logging
logger = logging.getLogger("timing")

class TimingMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        t0 = time.perf_counter()
        response = self.get_response(request)
        ms = (time.perf_counter() - t0) * 1000
        if ms > 200: # log slow requests
            logger.warning(f"SLOW {request.method} {request.path} {ms:.0f}ms")
        response["X-Response-Time"] = f"{ms:.1f}ms"
        return response

```

2.2 Settings Architecture — Every Key Setting Explained


```
Python ...cont
```

```
}  
}  
  
# ■■■ Celery  
CELERY_BROKER_URL = config("REDIS_URL", default="redis://127.0.0.1:6379/0")  
CELERY_RESULT_BACKEND = "django-db" # store results in PostgreSQL  
CELERY_ACCEPT_CONTENT = ["json"]  
CELERY_TASK_SERIALIZER = "json"  
CELERY_RESULT_SERIALIZER = "json"  
CELERY_TIMEZONE = "Asia/Dubai"  
CELERY_TASK_TRACK_STARTED = True # status = STARTED before running  
CELERY_WORKER_PREFETCH_MULTIPLIER = 1 # fetch 1 task at a time (fair)
```

2.3 The ORM — Advanced Patterns

Component	Role
WSGI	Synchronous. One request at a time per thread. Cannot hold open connections. python manage.py runserver
ASGI	Asynchronous. Can handle thousands of concurrent connections. Supports WebSockets, SSE, long-polling
Daphne	ASGI server. Handles HTTP/1.1 and WebSocket upgrade. Passes to Django via ASGI interface.
Channels	Django extension that adds ASGI support and the channel layer (Redis pub/sub abstraction).
uvicorn	Alternative ASGI server. Faster than Daphne for HTTP. Daphne preferred for WebSocket stability.

```

Python

# vikrant/asgi.py – the routing brain
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from scraper.routing import websocket_urlpatterns

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "vikrant.settings")

application = ProtocolTypeRouter({
    # HTTP requests → normal Django views
    "http": get_asgi_application(),

    # WebSocket requests → Channels consumers
    # AuthMiddlewareStack gives consumers access to request.user
    "websocket": AuthMiddlewareStack(
        URLRouter(websocket_urlpatterns)
    ),
})

# scraper/routing.py
from django.urls import re_path
from scraper import consumers

websocket_urlpatterns = [
    re_path(r"ws/logs/(?P<run_id>\d+)/$", consumers.LogConsumer.as_asgi()),
    re_path(r"ws/pipeline/status/$", consumers.PipelineStatusConsumer.as_asgi()),
]

# Start command:
daphne -b 0.0.0.0 -p 8000 vikrant.asgi:application
# -b 0.0.0.0 binds to all interfaces (needed for WSL→Windows forwarding)

```

Database Mastery

PostgreSQL internals, performance, and administration

MASTER REFERENCE

3.1 Schema Deep Dive — Every Table, Column, and Index

```
SQL

-- Inspect the full schema from psql
psql -U vikrant_user -h 127.0.0.1 -d vikrant_db

\dt                                -- list all tables
\d+ scraper_scraperscript          -- full table with indexes
\d+ scraper_pipeline              run
\d+ scraper_scraperrun
\d+ scraper_logline

-- Check auto-created indexes (Django creates one per FK)
SELECT indexname, indexdef
FROM   pg_indexes
WHERE  schemaname = 'public'
ORDER BY tablename, indexname;

-- Row counts
SELECT relname, n_live_tup
FROM   pg_stat_user_tables
ORDER BY n_live_tup DESC;

-- Table sizes
SELECT relname,
       pg_size_pretty(pg_total_relation_size(oid)) AS total,
       pg_size_pretty(pg_relation_size(oid))      AS table,
       pg_size_pretty(pg_indexes_size(oid))       AS indexes
FROM   pg_class
WHERE  relkind = 'r' AND relname LIKE 'scraper_%'
ORDER BY pg_total_relation_size(oid) DESC;
```

Critical Indexes to Add (Not in Default Migrations)

SQL

```
-- LogLine grows very fast. These indexes are production-critical.

-- 1. Fetch logs for a run, sorted by time (most common query)
CREATE INDEX CONCURRENTLY idx_logline_run_timestamp
ON scraper_logline (run_id, timestamp DESC);

-- 2. Find recent pipeline runs
CREATE INDEX CONCURRENTLY idx_pipelinerun_created
ON scraper_pipelinerun (created_at DESC);

-- 3. Filter runs by status
CREATE INDEX CONCURRENTLY idx_scraperrun_status
ON scraper_scraperrun (status, started_at DESC)
WHERE status IN ('running', 'pending'); -- partial index

-- CONCURRENTLY means PostgreSQL builds the index without locking writes
-- Always use CONCURRENTLY on production tables

-- Verify index is being used:
EXPLAIN ANALYZE
SELECT * FROM scraper_logline
WHERE run_id = 42
ORDER BY timestamp DESC LIMIT 100;
-- Look for "Index Scan using idx_logline_run_timestamp"
```

3.2

Query Performance — EXPLAIN ANALYZE Mastery

EXPLAIN ANALYZE is your primary performance tool. Read every output line until it makes sense.

EXPLAIN Term	What It Means
Seq Scan	Full table scan. Every row read. Acceptable for small tables (<10k rows). Bad at scale.
Index Scan	Uses a B-tree index. Reads only matching rows. Use this at scale.
Index Only Scan	All needed columns in the index itself. Fastest possible read.
Bitmap Heap Scan	Combines index results, then fetches rows. Used for lower-selectivity queries.
Hash Join	Joins two sets by building a hash table. Good for large joins with no useful index.
Nested Loop	For each row in outer, look up inner. Fast when inner side is small/indexed.
cost=X..Y	Planner estimate. X=startup cost, Y=total cost. Unitless — compare relative values.
actual time=X..Y	Real measured time in ms. Compare to cost to see if planner estimates are accurate.
rows=N	Actual rows returned. Large difference from plan estimate means stale statistics.
buffers: hit=N	Pages read from shared_buffers (RAM). Higher=better. Miss=disk read.

```
SQL

-- Run EXPLAIN ANALYZE BUFFERS for maximum information
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
  SELECT l.level, l.message, l.timestamp
  FROM   scraper_logline l
  WHERE  l.run_id = 1
  ORDER BY l.timestamp DESC
  LIMIT 100;

-- Update statistics (run if estimates are way off)
ANALYZE scraper_logline;

-- Find all slow queries (requires pg_stat_statements extension)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT query,
       calls,
       ROUND(mean_exec_time::numeric, 2) AS avg_ms,
       ROUND(total_exec_time::numeric, 2) AS total_ms,
       rows / calls AS avg_rows
FROM   pg_stat_statements
WHERE  mean_exec_time > 50    -- queries averaging > 50ms
ORDER BY total_exec_time DESC
LIMIT 20;
```

3.3 Transactions, Locks, and Concurrency

```
-- ■ Transactions ████████████████████████████████████████████████████████
-- Atomic: either ALL of these succeed, or NONE do
BEGIN;
    UPDATE scraper_scraperrun SET status = 'done', records = 150
        WHERE id = 42;

    UPDATE scraper_pipeline_run SET status = 'done', ended_at = NOW()
        WHERE id = (SELECT pipeline_id FROM scraper_scraperrun WHERE id = 42);

    INSERT INTO scraper_logline (run_id, level, message)
        VALUES (42, 'INFO', 'Completed: 150 records');
COMMIT;

-- On any error, roll back all changes:
-- ROLLBACK;

-- ■ Check for locks (useful during deadlock debugging) ██████████
SELECT pid, query, state, wait_event_type, wait_event
FROM pg_stat_activity
WHERE state != 'idle';

-- Find blocking queries:
SELECT blocked.pid, blocked.query,
       blocking.pid AS blocking_pid, blocking.query AS blocking_query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
      ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;

-- Kill a blocking query (use with care):
SELECT pg_cancel_backend(pid); -- polite (sends interrupt)
SELECT pg_terminate_backend(pid); -- forceful (kills connection)
```

3.4 Backup, Restore & Maintenance

[illegible][illegible]

PART

4

Redis & Messaging

The communication backbone of the entire system

MASTER REFERENCE

4.1 Redis Data Structures — How Celery and Channels Use Them

Structure	Use Case in Vikrant
STRING	Simple key→value. Used for: distributed locks, rate limiters, session data, cached results.
LIST	Ordered sequence. Celery task queue is a Redis LIST. R PUSH to add, BLPOP to consume.
HASH	Field→value map within one key. Celery stores task metadata in hashes.
SET	Unique unordered collection. Channel groups use sets to track member channel names.
ZSET	Sorted set with scores. Celery Beat uses this to schedule future tasks by timestamp.
PUBSUB	Publish/subscribe channels. Django Channels uses this for group_send broadcast.
STREAM	Append-only log. Alternative to PubSub for guaranteed delivery (not used here, but worth knowing).

```

bash

# ■■ Watch Redis in real time during a pipeline run ■■■■■■■■■■
redis-cli MONITOR
# You will see every command. Learn to recognise:
#   RPUSH celery ...      ← Django queuing a Celery task
#   BLPOP celery ...      ← worker picking up the task
#   PUBLISH asgi:...      ← Channels broadcasting to WebSocket clients
#   HSET celery-task-meta.. ← Celery storing task result/status

# ■■ Celery queue inspection ■■■■■■■■■■
redis-cli LLEN celery      # tasks waiting in default queue
redis-cli LRANGE celery 0 4  # first 5 queued tasks

# ■■ Channel layer inspection ■■■■■■■■■■
redis-cli KEYS "asgi:*"    # all Channels keys
redis-cli KEYS "asgi:group:run_logs_*" # specific run log channels

# ■■ Distributed lock (rate limiting pattern) ■■■■■■■■■■
# Acquire lock: SET if Not eXists with Expiry
redis-cli SET pipeline:lock "running" NX EX 3600
# Returns OK if lock acquired, nil if already held

# Release lock:
redis-cli DEL pipeline:lock

# ■■ Memory diagnostics ■■■■■■■■■■
redis-cli INFO memory
redis-cli MEMORY USAGE celery      # bytes used by one key
redis-cli --bigkeys                # find the largest keys

# ■■ Pub/Sub deep test ■■■■■■■■■■
# Terminal 1:
redis-cli SUBSCRIBE "run_logs_1"
# Terminal 2:
redis-cli PUBLISH "run_logs_1" '{"type":"log","level":"INFO","message":"test"}'

```

4.2 Channel Layer Architecture

The channel layer is the abstraction that lets Django Channels consumers and Celery tasks communicate without knowing about each other. Celery publishes to a group name. Any consumer subscribed to that group receives the message.

5.1 Celery App Factory & Configuration

[illegible]

5.2 Task Anatomy — Every Decorator Option

```

Python

# scraper/tasks.py – fully annotated task

@shared_task(
    bind=True,                # self = task instance (access self.retry, self.request)
    name="scraper.run",       # explicit name (default: module.function)
    max_retries=2,            # retry up to 2 times (3 total attempts)
    default_retry_delay=30,    # seconds to wait between retries
    soft_time_limit=600,      # sends SoftTimeLimitExceeded after 10min
    time_limit=660,           # hard kills after 11min
    acks_late=True,           # acknowledge AFTER completion (safe for crashes)
    reject_on_worker_lost=True, # re-queue if worker dies mid-task
)

def run_scraper_task(self, run_id: int, script_filename: str) -> dict:
    """Execute one scraper script and stream output to WebSocket."""
    from scraper.models import ScraperRun
    from datetime import datetime

    # Update DB status
    run = ScraperRun.objects.get(pk=run_id)
    run.status = "running"
    run.started_at = datetime.now()
    run.task_id = self.request.id
    run.save(update_fields=["status", "started_at", "task_id"])

    push_status(run_id, "running")

    try:
        records = execute_script(script_filename, run_id)
        run.status = "done"
        run.records = records
        run.ended_at = datetime.now()
        run.save(update_fields=["status", "records", "ended_at"])
        push_status(run_id, "done")
        return {"run_id": run_id, "records": records, "status": "done"}

    except SoftTimeLimitExceeded:
        run.status = "timeout"; run.save(update_fields=["status"])
        push_log(run_id, "ERROR", "Script exceeded 10-minute time limit")
        raise

    except Exception as exc:
        push_log(run_id, "ERROR", f"Exception: {exc}")
        try:
            raise self.retry(exc=exc)
        except self.MaxRetriesExceededError:
            run.status = "error"; run.save(update_fields=["status"])

```

```

Python ...cont

    push_status(run_id, "error")
    raise

```

5.3 The Full Pipeline Workflow

```

Python
# scraper/tasks.py - launch_full_pipeline

def launch_full_pipeline(triggered_by="manual"):
    """
    1. Create DB records for this pipeline run
    2. Dispatch: 7 scrapers in parallel → compiler → modem reboot
    3. Return pipeline_id immediately (non-blocking)
    """
    from scraper.models import ScraperScript, PipelineRun, ScraperRun

    # Get active scripts grouped by type
    scrapers = ScraperScript.objects.filter(is_active=True, is_compiler=False, is_modem=False)
    compiler = ScraperScript.objects.get(is_compiler=True, is_active=True)
    modem = ScraperScript.objects.get(is_modem=True, is_active=True)

    # Create pipeline run record
    pipeline = PipelineRun.objects.create(triggered_by=triggered_by)

    # Create individual run records
    scraper_runs = [
        ScraperRun.objects.create(pipeline=pipeline, script=s)
        for s in scrapers
    ]
    compiler_run = ScraperRun.objects.create(pipeline=pipeline, script=compiler)
    modem_run = ScraperRun.objects.create(pipeline=pipeline, script=modem)

    # Build workflow: group | compiler | modem
    from celery import group
    workflow = (
        group(
            run_scraper_task.s(r.id, r.script.filename)
            for r in scraper_runs
        )
        | run_compiler_task.s(compiler_run.id, compiler.filename)
        | run_modem_task.s(modem_run.id, modem.filename)
    )

    # Dispatch (non-blocking - returns a GroupResult)
    result = workflow.delay()

    # Save the group result ID for later revocation
    pipeline.task_group_id = result.id
    pipeline.save(update_fields=["task_group_id"])

    return {"pipeline_id": pipeline.id, "group_id": str(result.id)}

```

5.4 Worker Operations & Monitoring

[illegible]

6.1 WebSocket Protocol Deep Dive

Concept	Detail
Handshake	Client sends HTTP Upgrade request. Server responds 101 Switching Protocols. Connection stays open.
Frame format	Each message is a frame: FIN bit, opcode (text/binary/ping/pong/close), masking, payload.
Opcodes	0x1=text, 0x2=binary, 0x8=close, 0x9=ping, 0xA=pong. Ping/pong keep connections alive.
Masking	Client→Server frames are masked (XOR). Server→Client are unmasked. Prevents proxies caching.
Close codes	1000=normal, 1001=going away, 1006=abnormal (no close frame), 1011=server error.
Max size	No protocol limit, but Django Channels uses 64KB text by default. Configurable.
Reconnection	WebSocket has no built-in reconnect. Client must implement: onclose → setTimeout → new WebSocket

6.2 Consumer Deep Dive — Every Method Explained

```

Python

# scraper/consumers.py – fully annotated

import json
from channels.generic.websocket import AsyncWebsocketConsumer
from channels.db import database_sync_to_async
from scraper.models import LogLine, ScraperRun

class LogConsumer(AsyncWebsocketConsumer):
    """
    Streams log lines to the browser for one ScraperRun.
    URL: ws://host/ws/logs/<run_id>/
    """

    async def connect(self):
        # Extract run_id from the URL pattern
        self.run_id = int(self.scope["url_route"]["kwargs"]["run_id"])
        self.group = f"run_logs_{self.run_id}"

        # Join the Redis pub/sub group
        await self.channel_layer.group_add(self.group, self.channel_name)

        # Accept the connection (MUST call this or browser gets 403)
        await self.accept()

        # Replay recent history for clients who connect after run starts
        await self._send_history()

    async def _send_history(self):
        """Send last 100 log lines from DB to newly connected client."""
        logs = await database_sync_to_async(
            lambda: list(LogLine.objects.filter(run_id=self.run_id)
                          .order_by("timestamp")[:100])
        )()
        for log in logs:
            await self.send(text_data=json.dumps({
                "type": "log",
                "level": log.level,
                "message": log.message,
            }))

    async def disconnect(self, close_code):
        # Leave the group – stop receiving messages
        await self.channel_layer.group_discard(self.group, self.channel_name)
        # close_code: 1000=normal, 1001=browser closed tab, 1006=connection lost

    async def receive(self, text_data):

```


6.3 WebSocket Client — Production-Grade JavaScript

```

JavaScript

// Reconnecting WebSocket with exponential backoff
// Handles all failure modes: server restart, network drop, tab restore

class PipelineSocket {
  constructor(pipelineId) {
    this.pipelineId = pipelineId;
    this.delay = 1000; // start: 1s
    this.maxDelay = 30000; // cap: 30s
    this.attempts = 0;
    this.ws = null;
    this.handlers = {};
    this.connect();
  }

  connect() {
    const url = `ws://${location.host}/ws/pipeline/status/`;
    this.ws = new WebSocket(url);

    this.ws.onopen = () => {
      console.log("[WS] Connected");
      this.delay = 1000; // reset backoff
      this.attempts = 0;
      this._emit("connected");
    };

    this.ws.onmessage = (e) => {
      try {
        const data = JSON.parse(e.data);
        this._emit(data.type, data);
      } catch (err) {
        console.error("[WS] Invalid JSON:", e.data);
      }
    };

    this.ws.onclose = (e) => {
      console.log(`[WS] Closed (code=${e.code}). Retry in ${this.delay}ms`);
      this._emit("disconnected", { code: e.code });
      if (e.code !== 1000) { // 1000 = intentional close
        setTimeout(() => this.connect(), this.delay);
        this.delay = Math.min(this.delay * 2, this.maxDelay);
        this.attempts++;
      }
    };

    this.ws.onerror = (e) => console.error("[WS] Error:", e);
  }
}

```

```
on(type, fn) { this.handlers[type] = fn; }  
_emit(t, d) { if (this.handlers[t]) this.handlers[t](d); }  
close()     { this.ws.close(1000); }  
}
```

```
// Usage
```

```
const socket = new PipelineSocket(pipelineId);  
socket.on("log",      data => appendToTerminal(data));  
socket.on("status",   data => updateStagePanel(data));  
socket.on("connected", () => showStatus("Live"));  
socket.on("disconnected", () => showStatus("Reconnecting..."));
```

PART

7

Frontend Engineering

State machines, async patterns, and the terminal renderer

MASTER REFERENCE

7.1

LogCapture — Complete Implementation

```

Python
# scraper/tasks.py - LogCapture and execute_script

import io, sys, re, time

class LogCapture(io.RawIOBase):
    """
    Replaces sys.stdout during script execution.
    Every print() call is intercepted line by line.
    Each complete line is classified and pushed to Redis.
    """

    RECORD_RE = re.compile(r'(\d+)\s*(?:records?|leads?|rows?|users?)', re.I)

    def __init__(self, run_id: int, db_run):
        self.run_id = run_id
        self.db_run = db_run
        self._buf = ""
        self.records = 0

    def write(self, data) -> int:
        if isinstance(data, bytes):
            data = data.decode("utf-8", errors="replace")
        self._buf += data
        # Process complete lines immediately
        while "\n" in self._buf:
            line, self._buf = self._buf.split("\n", 1)
            if line.strip():
                self._process(line)
        return len(data)

    def _process(self, line: str) -> None:
        level = self._classify(line)
        # Persist to DB
        LogLine.objects.create(run_id=self.run_id, level=level, message=line)
        # Stream to browser
        push_log(self.run_id, level, line)
        # Extract record count for live counter
        m = self.RECORD_RE.search(line)
        if m:
            self.records = int(m.group(1))
            push_status(self.run_id, "running", records=self.records)

    @staticmethod
    def _classify(line: str) -> str:
        low = line.lower()
        if any(w in low for w in ["error", "exception", "traceback", "failed", "fatal"]): return "ERROR"

```

```

Python ...cont

    if any(w in low for w in ["warning", "warn", "slow", "timeout", "retry"]):
        return "WARN"
    if any(w in low for w in ["done", "success", "complete", "saved", "pushed"]):
        return "SUCCESS"
    return "INFO"

def flush(self): pass # required by IO protocol
def readable(self): return False
def writable(self): return True

def execute_script(script_filename: str, run_id: int) -> int:
    """Read, patch paths, and exec a scraper script with stdout capture."""
    import os
    from scraper.models import ScraperRun

    script_path = os.path.join(settings.SCRIPTS_DIR, script_filename)
    if not os.path.isfile(script_path):
        raise FileNotFoundError(f"Script not found: {script_path}")

    with open(script_path, "r", encoding="utf-8") as f:
        source = f.read()

    # Patch hardcoded Windows paths to project-relative Linux paths
    source = source.replace(
        r"C:\Users\Akhil\Desktop\VikrantDashboard",
        str(settings.BASE_DIR)
    )

    run = ScraperRun.objects.get(pk=run_id)
    capture = LogCapture(run_id, run)
    old_stdout, old_stderr = sys.stdout, sys.stderr
    sys.stdout = capture
    sys.stderr = capture

    try:
        code_obj = compile(source, script_path, "exec")
        exec(code_obj, {"__name__": "__main__", "__file__": script_path})
    finally:
        sys.stdout = old_stdout # ALWAYS restore, even if exec() raises
        sys.stderr = old_stderr

    return capture.records

```

7.2 Frontend State Machine & Terminal Renderer

```

JavaScript

// Pipeline state machine + terminal renderer

const STATE = {
  IDLE:"idle", STARTING:"starting", RUNNING:"running",
  DONE:"done", ERROR:"error", STOPPED:"stopped"
};

let current = STATE.IDLE;

const TRANSITIONS = {
  [STATE.IDLE]: [STATE.STARTING],
  [STATE.STARTING]: [STATE.RUNNING, STATE.ERROR],
  [STATE.RUNNING]: [STATE.DONE, STATE.ERROR, STATE.STOPPED],
  [STATE.DONE]: [STATE.IDLE],
  [STATE.ERROR]: [STATE.IDLE],
  [STATE.STOPPED]: [STATE.IDLE],
};

function go(to) {
  if (!TRANSITIONS[current]?.includes(to)) {
    console.warn(`Invalid transition ${current} → ${to}`); return;
  }
  current = to; renderState(to);
}

function renderState(s) {
  const btn_start = document.getElementById("btn-start");
  const btn_stop = document.getElementById("btn-stop");
  const label = document.getElementById("status-label");
  const map = {
    [STATE.IDLE]: {start:true, stop:false, label:"Ready", cls:""},
    [STATE.STARTING]: {start:false, stop:false, label:"Starting...", cls:"pulse"},
    [STATE.RUNNING]: {start:false, stop:true, label:"Running", cls:"pulse"},
    [STATE.DONE]: {start:true, stop:false, label:"Completed", cls:"done"},
    [STATE.ERROR]: {start:true, stop:false, label:"Error", cls:"error"},
    [STATE.STOPPED]: {start:true, stop:false, label:"Stopped", cls:""},
  }[s];
  btn_start.disabled = !map.start;
  btn_stop.disabled = !map.stop;
  label.textContent = map.label;
  label.className = map.cls;
}

// Terminal renderer
const LEVEL_CSS = {
  ERROR: "color:#EF4444;font-weight:bold",

```

```
WARN:    "color:#F59E0B",
SUCCESS: "color:#10B981;font-weight:bold",
INFO:    "color:#C9D1D9",
};

function tLog(level, message) {
  const term = document.getElementById("terminal");
  const line = document.createElement("div");
  const ts    = new Date().toLocaleTimeString().slice(0,8);
  const css   = LEVEL_CSS[level] || LEVEL_CSS.INFO;
  line.innerHTML = `[${ts}]</span> ` +
    `${escHtml(message)}</span>`;
  term.appendChild(line);
  term.scrollTop = term.scrollHeight; // auto-scroll
  if (term.children.length > 2000) term.removeChild(term.firstChild); // cap
}

function escHtml(s) { // prevent XSS from log content
  return s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");
}
```

Security & Hardening

Every vulnerability class and how to prevent it

MASTER REFERENCE

8.1 Security Configuration Checklist

PRODUCTION SECURITY BASELINE

Every item below must be verified before exposing this project to a network beyond localhost. Missing any one of these is a potential incident.

Item	Requirement
<code>SECRET_KEY</code>	Generated with <code>secrets.token_urlsafe(50)</code> . Stored in <code>.env</code> . Never committed.
<code>DEBUG=False</code>	Must be False in production. Debug mode leaks source code and stack traces.
<code>ALLOWED_HOSTS</code>	Explicit list only. Never <code>["*"]</code> in production.
<code>.env</code> permissions	<code>chmod 600 .env</code> — owner read/write only. No group/world access.
DB password	Strong random password. Not in code. Rotated periodically.
CSRF protection	Never disable <code>@csrf_exempt</code> on state-changing views accessed by users.
SQL injection	Use ORM or parameterised queries. Never f-string interpolation in raw SQL.
XSS prevention	Always use <code>escHtml()</code> before inserting log content into <code>innerHTML</code> .
Redis binding	bind 127.0.0.1 in <code>redis.conf</code> . Never expose Redis to external network.
PostgreSQL auth	<code>pg_hba.conf</code> : md5 or scram-sha-256. Never trust for remote connections.
Script isolation	Scripts <code>exec()</code> in the worker process. Malicious script = code execution. Review scripts before addin
Secrets in logs	Never log passwords, tokens, or API keys. Scrub them in <code>LogCapture._classify</code> .

8.2 The `exec()` Security Boundary

CRITICAL UNDERSTANDING

`execute_script()` runs arbitrary Python code in the Celery worker process with full system access. This is intentional — the scripts need to launch Chrome, access the filesystem, and make HTTP calls. The security boundary is: only trusted scripts in the `scripts/` directory are ever `exec()`'d. Adding a script to the database or `scripts/` folder is a privileged operation.

9.1 Performance Profiling Toolkit

[illegible]

9.2 Database Performance Tuning

```
SQL

-- Key postgresql.conf settings for a development/small production machine
-- Edit: sudo nano /etc/postgresql/14/main/postgresql.conf

-- Memory (adjust based on your RAM)
shared_buffers = 256MB          -- 25% of RAM is a good starting point
effective_cache_size = 768MB    -- estimate of OS file cache (75% of RAM)
work_mem = 4MB                  -- per-sort/hash operation (be careful with concurrent queries)
maintenance_work_mem = 64MB     -- for VACUUM, CREATE INDEX

-- WAL (write-ahead log)
wal_buffers = 16MB
checkpoint_completion_target = 0.9

-- Query planner
random_page_cost = 1.1          -- for SSD storage (default 4.0 assumes HDD)
effective_io_concurrency = 200  -- for SSD

-- Connection pooling (if not using pgBouncer)
max_connections = 100

-- Logging slow queries
log_min_duration_statement = 100 -- log queries taking > 100ms
log_line_prefix = '%t [%p]: '

-- Apply without restart:
sudo -u postgres psql -c "SELECT pg_reload_conf();"
```

9.3 Scaling Bottlenecks & Solutions

Bottleneck	Resolution
LogLine table too large	Archive logs older than 30 days. Add created_at index. Consider pg_partman for automatic monthly partitioning.
Too many DB connections	Add pgBouncer connection pooler. Set CONN_MAX_AGE=60 in Django. Reduce Celery concurrency.
Celery tasks queuing up	Increase --concurrency. Add more workers. Check if tasks are blocking on network (Chrome, API calls).
Redis memory growing	Set maxmemory-policy allkeys-lru in redis.conf. Clear old Celery result keys with celery -A vikrant flushall.
WebSocket connections drop	Check daphne process RAM. Add CHANNEL_LAYERS capacity config. Consider adding a load balancer.
Chrome processes leaking	Ensure scripts call driver.quit() in finally block. Add periodic: pkill -f chromedriver
Slow scraper API calls	Add request timeout in scripts. Log slow calls. Consider retry with backoff in the script itself.

Troubleshooting Bible

Every known failure mode with root cause and fix

MASTER REFERENCE

HOW TO USE THIS SECTION

Find the symptom in the section header. Read ROOT CAUSE before applying the FIX.

Understanding why something breaks prevents the same problem recurring. If you see a symptom not listed here, diagnose it, then add it to this document.

10.1 Service Startup Failures

SYMPTOM

Dashboard returns 502 Bad Gateway

CRITICAL

ROOT CAUSE

Daphne is not running, or crashed on startup. Port 8000 is not listening.

FIX

Check: `ss -tlnp | grep :8000`. If empty, restart Daphne: `daphne -b 0.0.0.0 -p 8000 vikrant.asgi:application`. Check `logs/daphne.log` for the crash reason.

SYMPTOM

Terminal shows "Cannot connect to backend"

HIGH

ROOT CAUSE

WebSocket connection failed. Either Daphne is down, or the `ws://` URL is wrong.

FIX

Open browser DevTools → Network → WS. Check the WebSocket connection URL and response code. Verify Daphne is running and ASGI routing is correct.

SYMPTOM

`redis.exceptions.ConnectionError`

CRITICAL

ROOT CAUSE

Redis is not running. Common after WSL restart.

FIX

`sudo service redis-server start && redis-cli ping`. Add `start-services` alias to your workflow. Check `/var/log/redis/redis-server.log` if redis fails to start.

SYMPTOM

django.db.OperationalError: could not connect

CRITICAL**ROOT CAUSE**

PostgreSQL not running, wrong host/password in settings, or DB does not exist.

FIX

sudo service postgresql start. Check .env DB_HOST=127.0.0.1. Verify: psql -U vikrant_user -h 127.0.0.1 -d vikrant_db. Check pg_hba.conf allows md5 for 127.0.0.1.

SYMPTOM

ModuleNotFoundError: No module named django

HIGH**ROOT CAUSE**

Virtual environment is not activated in this terminal session.

FIX

source \$PROJ/venv/bin/activate. Verify: which python shows venv path. Add "proj" alias to ~/.bashrc.

SYMPTOM

celery.exceptions.NotRegistered: scraper.tasks.run_scraper_task

HIGH**ROOT CAUSE**

Celery cannot find the task. autodiscover_tasks not working, or __init__.py import missing.

FIX

Verify vikrant/__init__.py imports celery_app. Check vikrant/celery.py has app.autodiscover_tasks(). Restart worker after any task code change.

10.2**Pipeline Execution Failures****SYMPTOM**

Stage panels stuck in "pending" after clicking Start

CRITICAL**ROOT CAUSE**

Celery worker is not running, or task failed to dispatch. No tasks in the queue.

FIX

Check: celery -A vikrant inspect active. If no worker: start the worker. Check logs/celery_worker.log. Check Redis queue: redis-cli LLEN celery.

SYMPTOM

Logs stopped appearing mid-run

HIGH**ROOT CAUSE**

One of: Celery worker crashed, Redis pub/sub dropped, WebSocket disconnected, or script is hanging without printing.

FIX

Check: (1) celery inspect active — is the task still running? (2) redis-cli SUBSCRIBE run_logs_<id> and see if messages arrive. (3) Browser DevTools WS tab — is connection open? (4) Check log for last line printed.

SYMPTOM

Script fails with "Script not found"

MEDIUM**ROOT CAUSE**

Script file not in scripts/ directory, or filename in DB does not match actual file.

FIX

Is \$PROJ/scripts/. Compare with: python manage.py shell →
ScraperScript.objects.values_list("filename", flat=True). Copy missing script files to scripts/.

SYMPTOM

Compiler runs before all scrapers finish

HIGH**ROOT CAUSE**

Celery chord not configured correctly. group | chain broken by missing s() signature.

FIX

Verify run_compiler_task.s() uses .s() not .si(). Check chord structure in launch_full_pipeline. The | operator requires .s() signatures on all tasks.

SYMPTOM

Modem reboot fails silently

MEDIUM**ROOT CAUSE**

Chrome cannot reach 192.168.8.1 from WSL, or modem login credentials changed.

FIX

Test: curl http://192.168.8.1 from WSL. If unreachable, check WSL network route. If reachable, check modem password. Check modem_run log lines for Chrome errors.

SYMPTOM

ScraperRun stays in "running" after Celery worker crash

HIGH**ROOT CAUSE**

Worker killed mid-task. Without `acks_late=True`, task acknowledged before completion. DB never updated.

FIX

Add `acks_late=True`, `reject_on_worker_lost=True` to task decorator. Add a cleanup: `python manage.py shell → ScraperRun.objects.filter(status="running", started_at__lt=<2h ago>).update(status="error")`

SYMPTOM

0 records collected — no error shown

MEDIUM**ROOT CAUSE**

Script ran without exception but found no data. API returned empty, login failed silently, or JWT expired.

FIX

Read the log lines carefully — look for the login step. Check `affiliate.winfix.fun` is accessible. Verify JWT extraction succeeded. Run the script manually in a terminal to see full output.

10.3**WebSocket & Real-Time Failures****SYMPTOM**

WebSocket connects then immediately disconnects

HIGH**ROOT CAUSE**

Consumer raised an exception during `connect()`. Or channel layer not configured.

FIX

Check Daphne console output for the exception. Verify `CHANNEL_LAYERS` in settings. Check Redis is running. Add `try/except` around `connect()` body and log the error.

SYMPTOM

Logs appear in database but not in browser terminal

HIGH**ROOT CAUSE**

`push_log()` worked (DB has logs) but `pub/sub` message was not received by the consumer.

FIX

Check: `redis-cli SUBSCRIBE run_logs_<id>` in a separate terminal while pipeline runs. If messages appear there, the Channels consumer is not subscribing correctly. Check `group_add` call in `consumer.connect()`.

SYMPTOM

Late-joining browser sees no history

MEDIUM**ROOT CAUSE**

`_send_history()` not implemented or `LogLine` query filtering incorrectly.

FIX

Verify `consumer.connect()` calls `_send_history()`. Check `LogLine.objects.filter(run_id=...)` returns rows. Verify `database_sync_to_async` wrapper is used (consumer is async).

SYMPTOM

Multiple browsers cause duplicate log entries

LOW**ROOT CAUSE**

`LogLine.objects.create()` is called in `LogCapture._process()` which runs in the worker. Multiple browsers are subscribing fine — the issue is elsewhere.

FIX

If logs appear doubled in the terminal, check if the `LogConsumer.log_message` handler is registered multiple times. Check JS does not call `openWebSocket()` twice.

10.4**Diagnostic Command Reference**

[illegible]

Upgrade Playbooks

Step-by-step procedures for every type of change

MASTER REFERENCE

11.1 How to Add a New Scraper Script

ZERO CODE CHANGES REQUIRED

The dashboard loads scripts dynamically from the database. Adding a new scraper requires only placing the file in scripts/ and registering it in the database. No Python code changes, no migrations, no redeployment.

```
bash

# Step 1: Place the script file
cp /path/to/New_Panel_Scraper.py $PROJ/scripts/
chmod 750 $PROJ/scripts/New_Panel_Scraper.py

# Step 2: Verify it runs standalone (catch errors before adding to pipeline)
cd $PROJ && source venv/bin/activate
python scripts/New_Panel_Scraper.py # must complete without exception

# Step 3: Register in database
python manage.py shell
```

```
Python

from scraper.models import ScraperScript

script, created = ScraperScript.objects.get_or_create(
    filename="New_Panel_Scraper.py",
    defaults={
        "name": "New Panel Name",
        "is_compiler": False,
        "is_modem": False,
        "is_active": True,
        "order": 10,
    }
)
print("Created:", created, "| ID:", script.id)
```

```

bash

# Step 4: Verify it appears in the dashboard
# Open http://localhost:8000 → Scraper page
# New panel should appear with "pending" status

# Step 5: Run a single test
celery -A vikrant call scraper.tasks.run_scraper_task \
  --args='[<run_id>, "New_Panel_Scraper.py"]'
# Watch the terminal in the browser for live output

# Step 6: If script fails, check common issues:
# - Hardcoded Windows paths (replace with settings.BASE_DIR)
# - Chrome profile path (must be writable in WSL)
# - Different API parentId or credentials

```

11.2 Adding a Database Field — Safe Migration Process

```

bash

# NEVER modify production DB directly. Always use migrations.

# Example: add "gateway_id" field to ScraperScript

# Step 1: Edit models.py
# Add: gateway_id = models.CharField(max_length=50, blank=True, default="")

# Step 2: Create migration
python manage.py makemigrations scraper --name="add_gateway_id_to_scraperscript"

# Step 3: Inspect the generated migration
cat scraper/migrations/0002_add_gateway_id_to_scraperscript.py
# Verify: operations = [migrations.AddField(...)]
# Must be AddField with a default — never AddField without default on non-nullable

# Step 4: Test the migration on a copy first
pg_dump -U vikrant_user -h 127.0.0.1 -d vikrant_db -F c -f pre_migration.dump
python manage.py migrate scraper

# Step 5: Verify
psql -U vikrant_user -h 127.0.0.1 -d vikrant_db
\d scraper_scraperscript -- new column should appear

# Step 6: If migration fails, roll back
python manage.py migrate scraper 0001 -- revert to previous migration
# Then fix the model and try again

# Step 7: Update admin.py to show the new field
# Add "gateway_id" to list_display and possibly list_editable

```

11.3 Upgrading a Python Package — Safe Procedure

```
bash

# NEVER pip install --upgrade without a plan

# Step 1: Check what you have
pip show django

# Step 2: Read the changelog for the target version
# Django: https://docs.djangoproject.com/en/4.2/releases/
# Celery: https://docs.celeryq.dev/en/stable/changelog.html
# channels: https://github.com/django/channels/blob/main/CHANGELOG.txt

# Step 3: Check for incompatible changes
# Especially: deprecated APIs, removed features, settings changes

# Step 4: Test in isolation
python3 -m venv venv_test && source venv_test/bin/activate
pip install -r requirements.txt
pip install "django==4.2.8" # target version
python manage.py check
python manage.py test

# Step 5: If tests pass, update requirements.txt
pip freeze > requirements.txt

# Step 6: Remove test venv
deactivate && rm -rf venv_test

# Step 7: Install in your real venv
source venv/bin/activate
pip install -r requirements.txt
python manage.py check

# Step 8: Run a full test pipeline
daphne ... && celery ... && go to browser and test
```

11.4 Changing the Schedule — Celery Beat

```
bash

# Method 1: Django Admin (recommended – no code, takes effect immediately)
# Go to: http://localhost:8000/admin/django_celery_beat/periodictask/
# Click on "hourly-pipeline"
# Change the crontab or interval
# Save – Beat picks up the change within 5 seconds

# Method 2: Django shell (for scripting)
python manage.py shell
```

```
Python

from django_celery_beat.models import PeriodicTask, CrontabSchedule

# Change to "every 2 hours"
schedule, _ = CrontabSchedule.objects.get_or_create(
    minute="0", hour="*/2", day_of_week="*",
    day_of_month="*", month_of_year="*"
)

task = PeriodicTask.objects.get(name="hourly-pipeline")
task.crontab = schedule
task.save()

# Disable a task
task.enabled = False
task.save()

# Create a new scheduled task
PeriodicTask.objects.create(
    crontab=schedule,
    name="daily-cleanup",
    task="scraper.tasks.cleanup_old_logs",
    args="[]",
    enabled=True,
)
```

System Ownership

Monitoring, runbooks, and the architecture you must own completely

MASTER REFERENCE

12.1 The Complete Architecture Map

Component	Role
Browser	Chrome/Firefox. Opens <code>http://localhost:8000</code> . Connects two WebSockets. Sends <code>fetch()</code> calls.
Daphne	ASGI server. Accepts HTTP and WebSocket. Forks: HTTP→Django views, WS→Consumers.
Django	The application core. URL routing, views, ORM, admin, middleware, settings.
Django Channels	WebSocket layer. Consumers handle WS connections. Channel layer = Redis pub/sub bridge.
Celery Worker	Background process. Picks tasks from Redis. Runs scraper scripts. Publishes logs to Redis.
Celery Beat	Scheduler process. Reads <code>PeriodicTask</code> table. Dispatches tasks on schedule.
Redis	Two roles: (1) Celery task queue (LIST). (2) Channels pub/sub (PUBSUB).
PostgreSQL	Persistent storage: scripts, pipeline runs, scraper runs, log lines, task results, schedules.
<code>scripts/*.py</code>	9 Python scripts. Run inside Celery worker via <code>exec()</code> . Print to captured stdout.
<code>data/input/</code>	CSVs written by scraper scripts. Read by <code>Compiler.py</code> .
<code>data/repo/</code>	Backup CSV used by <code>Compiler.py</code> for deduplication.

12.2 Daily Operations Runbook

```
bash

#!/usr/bin/env bash
# daily_check.sh – Run every morning before using the dashboard

set -euo pipefail

log() { echo "[$(date +%H:%M:%S)] $*"; }
ok() { echo " [OK] $*"; }
err() { echo " [ERR] $*" >&2; }

log "=== Vikrant Daily Health Check ==="

# Services
pg_isready -h 127.0.0.1 -q && ok "PostgreSQL" || { err "PostgreSQL DOWN"; exit 1; }
redis-cli ping -q > /dev/null && ok "Redis" || { err "Redis DOWN"; exit 1; }

# Project
cd $PROJ && source venv/bin/activate
python manage.py check --deploy 2>&1 | grep -v "^System" | head -5

# Database row counts
psql -U vikrant_user -h 127.0.0.1 -d vikrant_db -t -c \
  "SELECT 'Pipeline runs:', COUNT(*) FROM scraper_pipeline_run",

psql -U vikrant_user -h 127.0.0.1 -d vikrant_db -t -c \
  "SELECT 'Log lines:', COUNT(*) FROM scraper_logline"

# Disk space (logs grow)
du -sh $PROJ/logs/ $PROJ/data/

# Large log files
find $PROJ/logs -name "*.log" -size +50M -exec ls -lh {} \;

log "=== Health check complete ==="
```

12.3 Key Settings Reference Card

Setting	Recommendation / Impact
CELERY_WORKER_CONCURRENCY	4. Number of parallel tasks. Increase for more scrapers. Limited by CPU cores and Chrome memory.
CELERY_TASK_SOFT_TIME_LIMIT	600. Seconds before SoftTimeLimitExceeded. Should exceed longest script runtime.
CELERY_TASK_TIME_LIMIT	660. Hard kill after this many seconds. Always > soft limit.
CONN_MAX_AGE	60. Django reuses DB connections for 60s. Reduces connection overhead.
CHANNEL_LAYERS capacity	Default 100 messages per channel. Increase if logs are dropped.
Redis maxmemory	Set to 512mb-1gb. With maxmemory-policy allkeys-lru to prevent OOM.
PostgreSQL shared_buffers	256MB for 1GB RAM. 25% of available RAM.
log_min_duration_statement	100ms. Log all queries slower than 100ms for analysis.
Scripts concurrency	7 scrapers run in parallel. Each launches Chrome. Needs 7+ GB RAM for Chrome.
Beat scheduler	DatabaseScheduler polls every 5 seconds. Change sync interval in beat --max-interval.

12.4 The Questions You Must Be Able to Answer Cold

SYSTEM OWNERSHIP TEST

If someone calls you at 2am and asks any of these, you must answer without hesitation. If you hesitate on any, that is your next study target.

Architecture

1. How does a log line printed by `Black_Hat_Meta.py` appear in the browser terminal? Trace every hop.
2. Why can't we use `python manage.py runserver` for production? What specifically breaks?
3. What happens if Redis goes down while a pipeline is running? What is lost? What survives?
4. What happens if the Celery worker process is killed while executing a scraper script?
5. Why does `launch_full_pipeline()` return immediately? How does the browser know when it is done?

Database

1. The `LogLine` table has 5 million rows and log queries are slow. What do you do, in order?
2. A migration failed halfway through. The DB is in an inconsistent state. How do you recover?
3. What does `EXPLAIN ANALYZE` tell you that `EXPLAIN` does not?
4. Two Celery workers try to update the same `ScraperRun` row simultaneously. What happens?
5. How do you find the top 5 slowest queries running against the project right now?

Celery

1. A scraper task is stuck in `STARTED` state for 3 hours. What are the possible causes? How do you investigate?
2. The compiler task is running before all 7 scrapers have finished. Why might this happen?
3. What is `acks_late=True` and why does it matter when a worker crashes mid-task?
4. How do you add a new Celery task without restarting Django? Without restarting the worker?
5. What is the difference between `celery.control.revoke` with `terminate=True` vs `terminate=False`?

Operations

1. How do you add a new scraper script to the pipeline with zero code changes?
2. How do you change the pipeline schedule from hourly to every 6 hours?
3. A developer wants to upgrade Django from 4.2 to 5.0. What is your process?
4. How do you safely add a new column to the `ScraperRun` table with the pipeline in use?
5. The Celery Beat process is running two copies. What went wrong and what breaks as a result?

Security

1. A developer accidentally committed `.env` to git. What do you do in the next 5 minutes?
2. What prevents a malicious script filename like `"../../etc/passwd"` from being executed?
3. Why is `exec()` in `execute_script()` not a security vulnerability given proper controls?
4. Where are the secrets in this project stored and how are they loaded at runtime?
5. What does `@csrf_exempt` do and which views in this project use it and why?

Final Note

This document is a living reference. Every time you encounter a new problem, add it to Part 10. Every time you perform an upgrade, document the exact steps in Part 11. Every time someone asks you a question you could not answer, write it in the ownership test. The goal is not to memorise this document — it is to understand the system so completely that this document becomes unnecessary.

THE MARK OF SYSTEM OWNERSHIP

You own a system when you can look at any failure, any metric, any piece of output, and immediately know what it means, what caused it, and what to do about it. You are there when the troubleshooting bible has no entry for the problem you are solving — because you diagnosed it yourself and added the entry.